Лекция 1
Лямбда-исчисление

01

Лямбда-исчисление

Минимальная формальная модель вычислений

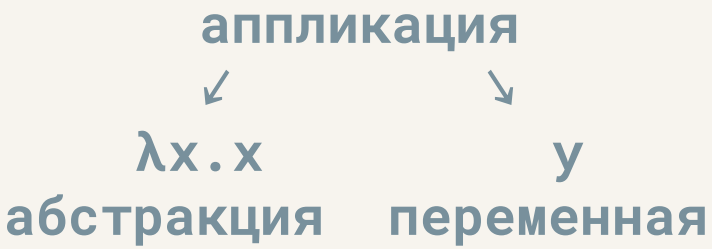
Термы λ-исчисления

$$M, N ::= x \quad | \quad \lambda x.M \quad | \quad M\ N$$

- x = переменная
- $\lambda x.M$ = абстракция
- $M\ N$ = аппликация

Синтаксическое дерево

```
(λx.x) y
```



Свободные переменные

$$FV(x) = \{x\}$$

$$FV(M\ N) = FV(M) \cup FV(N)$$

$$FV(\lambda x.M) = FV(M) \setminus \{x\}$$

Вопрос

$FV(\lambda y.y\ z) = ?$

$FV(\lambda x.w\ x\ (\lambda y.y\ z)) = ?$

Лексическое связывание

```
λx. x (λx.x)
```

- внешний λx связывает первое x
- внутренний λx затеняет внешний и связывает последнее x

α-эквивалентность и подстановка

Два терма альфа-эквивалентны, если один можно превратить в другой путем правильного переименования связанных переменных.

$$\lambda x.x \equiv_\alpha \lambda y.y$$

$$\lambda x.y \not\equiv_\alpha \lambda y.y$$

β-редукция = вычисление

$$(\lambda x.M)\ N \quad \rightarrow_\beta \quad M[x := N]$$

β-редукция: подстановка аргумента вместо параметра

Пример: β-редукция в один шаг

$$(\lambda x.x)\ y \rightarrow_\beta y$$

подставляем y вместо x

Пример: β-редукция в три шага

$$((\lambda x.\lambda y.x\ y)\ (\lambda z.z))\ a$$

$$\rightarrow_\beta\ (\lambda y.(\lambda z.z)\ y)\ a$$

$$\rightarrow_\beta\ (\lambda z.z)\ a$$

$$\rightarrow_\beta\ a$$

Redex и нормальная форма

β-redex: (λx.M) N

```
(λx.y x) z – есть β-redex
(λx.y x)   – нет β-redex
(y x)      - нет β-redex
```

β-нормальная форма

M находится в β-нормальной форме
⇔
в M нет подтерма вида $(\lambda x.P)\ Q$

$(\lambda x.x)$ — нормальная форма

$(\lambda x.x)\ y$ — нет, так как содержит β-redex

Ω: вычисление без нормальной формы

$$\Omega = (\lambda x.x\ x)(\lambda x.x\ x)$$

$$\Omega \rightarrow_\beta \Omega \rightarrow_\beta \Omega \rightarrow_\beta \ldots$$

02

Расширения λ-исчисления

Как минимальная модель становится практическим языком?

Константы и примитивы

$$M ::= x \mid \lambda x.M \mid M\ N \mid c$$

c: числа, true/false, +, *,...

δ-редукция

δ-редукция — вычисление встроенного примитива

$(+\ 2\ 3) \to_\delta 5$

$(*\ 6\ 7) \to_\delta 42$

β-reduction + δ-reduction

$$(\lambda x.\ x + 1)\ 41$$
$$\rightarrow_{\beta}\ 41 + 1$$
$$\rightarrow_{\delta}\ 42$$

1, 41, 42 — константы расширенного λ-исчисления

Именованные термы

λ-исчисление		Clojure
`id ≡ λx.x` `(id 42)`	→	`(def id (fn [x] x))` `(id 42) ;; => 42`

`foo ≡ λx.λy.x`		`(def foo`
		`  (fn [x]`
		`  (fn [y] x)))`

let: локальное имя

```
let x = N in M
```

означает:
в теле M имя x
обозначает N

λ-запись:
(λx.M) N

```
(λx.x + x) 10
```

```
(let [x 10]
  (+ x x))

;; => 20
```

Рекурсия: ссылка на себя

fact вызывает fact

```
fact(n) =
  if n = 0 then 1
  else n · fact(n - 1)
```

проблема:
функция использует собственное имя

Убираем самоссылку

Передадим рекурсивный вызов
как параметр f

```
F = λf.λn.(if (zero? n) 1 (* n (f (- n 1))))
```

F — не factorial,
а генератор factorial

Неподвижная точка

Хотим получить функцию fact, для которой:

```
fact = F(fact)
```

Неподвижная точка F — такое x,
что F(x) = x

Комбинатор Y: Y F = F (Y F)

Как работает YF

$$Y = \lambda f.(\lambda x.f(x\ x))(\lambda x.f(x\ x))$$

Применим Y к F:

$$Y\ F \rightarrow_\beta (\lambda x.F(x\ x))(\lambda x.F(x\ x))$$

Обозначим:

$$A = \lambda x.F(x\ x)$$

Тогда $Y\ F = A\ A$

Почему это даёт рекурсию

Так как A = λx.F(x x), то

A A → λx.F(x x) A →β F(A A)

Но A A = Y F, значит:

Y F →β F(Y F)

Y F — неподвижная точка F

03

Порядок вычислений

Когда redex несколько, какой сокращать первым?

Частичный порядок редукций

```
(λx.x) ((λy.y) z)
```

внешний redex:

```
(λx.x) ((λy.y) z)
M := x, N := ((λy.y) z)
```

внутренний redex:

```
(λy.y) z
M := y, N := z
```

оба пути дают z

Стратегия влияет на завершаемость

$$(\lambda x.\ \lambda z.z)\ \Omega$$

$$\Omega = (\lambda x.x\ x)(\lambda x.x\ x)$$

Вариант 1: **Нормальный порядок**
сначала внешний redex:
$(\lambda x.\ \lambda z.z)\ \Omega \ \rightarrow_\beta\ \lambda z.z$
Ω не вычисляется

Вариант 2: **Аппликативный порядок**
сначала вычисляем аргумент:
$\Omega \rightarrow_\beta \Omega \rightarrow_\beta \Omega \rightarrow \ldots$
до внешнего redex не доходим

Варианты отложенного вычисления

normal order — редуцируем самый левый внешний redex

call-by-name — аргумент не вычисляется до использования; при повторном использовании вычисляется заново

call-by-need — аргумент не вычисляется до использования; после первого вычисления результат запоминается

Call-by-name: пример

```
(λx. x + x) (1 + 2)
→β (1 + 2) + (1 + 2)
→δ 3 + 3
→δ 6
```

Аргумент подставляется в тело без предварительного вычисления.
Если x используется дважды, выражение (1 + 2) появляется два раза.

Call-by-need: пример

```
let x = (1 + 2)
in  x + x
```

x вычисляется один раз:

$1 + 2 \to_{\delta} 3$

затем: $3 + 3 \to_{\delta} 6$

Call-by-need = отложить вычисление до первого использования + запомнить результат.
Поэтому повторное использование x не повторяет исходное вычисление.

04

λ-исчисление → Clojure

Предскажите → объясните → запустим REPL

λ-исчисление ↔ Clojure

λx.M	↔	(fn [x] M)
M N	↔	(M N)
β-reduction	↔	function evaluation
lexical binding	↔	lexical scope / closure

Что вернёт REPL?

```
((fn [x]
   (fn [y] (+ x y)))
 10)
```

10 **функция** **ошибка**

Функция как результат вычисления

```
((fn [x]
   (fn [y] (+ x y)))
 10)
```

⇒ функция

(λx.λy.x+y) 10 →β λy.10+y

Что вернёт REPL?

```
(def f
  (let [x 10]
    (fn [y] (+ x y))))

(let [x 1000]
  (f 5))
```

15 **105** **1005**

Лексический контекст: ответ

```
(def f
  (let [x 10]
    (fn [y] (+ x y))))

(let [x 1000] (f 5))
```

⇒ **15**

замыкание сохраняет
лексическое окружение
создания функции

Что вернёт REPL?

```
((fn [x] 10)
 (boom))
```

10 **BOOM! + 10** **ошибка**

Аппликативный порядок

```
(defn boom []
  (println "BOOM!")
  42)

((fn [x] 10) (boom))
```

```
BOOM!
⇒ 10
```

аргумент вычисляется
до применения функции

Clojure: delay / force

```
(def x
  (delay
    (do
      (println "compute")
      42)))
```

```
(force x)
;; compute
 ;; => 42

(force x)
 ;; => 42
```

call-by-need:
defer + memoize

Clojure не lazy language

```
(def xs
  (map expensive-fn
    (range    1000000)))
```

обычная аппликация — strict/eager

ленивость живёт в абстракциях: LazySeq, map/filter, delay/force

ISeq ≠ lazy сам по себе

Ω в Clojure

```
((fn [x] 42)
   ((fn [x] (x x))
    (fn [x] (x x))))
```

x не используется,
но Clojure сначала
пытается вычислить Ω

→ вычисление не
завершается

Почему if — special form?

```
(if true
    42
    (/ 1 0))
```

⇒ 42

невыбранная ветка
не вычисляется

обычная функция так не
умеет

quote: код как данные

```
'(+ 1 (* 2 3))
```

```
⇒ (+ 1 (* 2 3))

(first ...)  ⇒ +
(last ...)   ⇒ (* 2 3)
(eval ...)   ⇒ 7
```